

Đề xuất cải tiến HermesQA sau khi khảo sát Open Code Review của Alibaba

Ngày: 05/10/2026 · **Trạng thái:** Đã triển khai cả 7 đề xuất (06/10/2026, commit `d2c5a0b`); số liệu đo lại sẽ được bổ sung ở mục 9 khi hoàn tất.

1. Tóm tắt một trang

HermesQA là "trợ lý soát lỗi code" tự động: khi lập trình viên gửi một thay đổi (gọi là *pull request*, viết tắt PR), hệ thống đọc thay đổi đó, tìm lỗi và ghi nhận xét vào đúng dòng. Nó kết hợp hai nguồn: **công cụ kiểm tra tĩnh** (phần mềm quét theo quy tắc cố định, chạy trong "hộp cát" cách ly) và **mô hình ngôn ngữ lớn** (AI đọc hiểu ngữ cảnh).

Open Code Review (viết tắt OCR) là công cụ cùng mục đích do Alibaba dùng nội bộ hai năm rồi mở mã nguồn năm 2026, hiện có hơn 43.000 sao trên GitHub. Chúng tôi đã đọc kỹ mã nguồn, các lời nhắc gửi cho AI, bộ lọc file, cơ chế định vị nhận xét, và bài báo đánh giá AACR-Bench của họ.

Kết luận chính:

- OCR **không hề có công cụ kiểm tra tĩnh**. Phần "quy tắc có sẵn" của họ là các bản danh sách kiểm tra bằng văn bản đưa vào lời nhắc cho AI. Sandbox kiểm tra tĩnh của HermesQA vì thế vẫn là điểm khác biệt thật, và thực nghiệm của đồ án đã cho thấy nó bắt được lỗi mà AI bỏ sót.
- OCR mạnh ở chỗ HermesQA yếu: **những bước làm bằng quy tắc cứng, không giao cho AI** – chọn file nào cần xem, ghim nhận xét vào đúng dòng, và lọc kết quả. Họ gọi đây là "kỹ thuật tắt định".
- Bảy đề xuất bên dưới đều mượn ý tưởng từ OCR nhưng nhỏ, đo được bằng bộ thực nghiệm sẵn có của đồ án, và không cần viết lại hệ thống. Ba đề xuất đầu có thể làm trong vài ngày và trực tiếp trả lời hai điểm yếu đã lộ ra trong thực nghiệm (bước tự kiểm chứng không có tác dụng, và tỷ lệ báo động giả cao).

2. Bối cảnh: hai hệ thống làm cùng một việc theo hai cách

Hãy tưởng tượng việc soát một bản thảo trước khi in.

- HermesQA** thuê ba biên tập viên AI, mỗi người một chuyên môn (logic chương trình, kiểm thử, hạ tầng). Mỗi người nhận bản thảo một lần, đọc, viết nhận xét kèm số trang và số dòng. Trước đó, một bộ máy quét chính tả tự động (công cụ tĩnh) đã chạy và liệt kê lỗi theo quy tắc. Sau cùng, hệ thống gộp hai nguồn và đăng lên.
- OCR** thuê một biên tập viên AI nhưng cho người đó quyền **lật giở cả cuốn sách**: mở trang khác, tìm từ khóa, xem bản thảo chương liên quan. Trước khi giao việc, một thư ký (quy tắc cứng) đã quyết định chương nào không cần xem (phụ lục, mục lục tự sinh, file chứa mật khẩu), chia bản thảo thành từng cụm chương liên quan để nhiều biên tập viên làm song song. Khi biên tập viên viết nhận xét, họ

không được ghi số dòng, mà phải chép nguyên văn vài dòng cần sửa; thư ký sẽ tự tìm vị trí trong bản thảo. Cuối cùng một người kiểm tra chéo chỉ loại bỏ nhận xét nào bị **chứng minh là sai**, còn nghi ngờ thì giữ.

Bảng dưới đây đặt hai cách làm cạnh nhau. Cột cuối giải thích ý nghĩa thực tế.

Khía cạnh	OCR	HermesQA hiện tại	Ý nghĩa với người dùng
Công cụ kiểm tra tĩnh	Không có	5 công cụ trong hộp cát cách ly mạng	HermesQA bắt được một số lỗi lặp lại y hệt mỗi lần chạy , điều AI không bảo đảm
AI đọc gì	Thay đổi + được quyền tự mở file khác, tìm kiếm trong toàn bộ mã nguồn	Thay đổi + vài dòng xung quanh, không có quyền đọc thêm	OCR hiểu ngữ cảnh rộng hơn nhưng tốn công xây dựng gấp nhiều lần
Ghim nhận xét vào dòng	AI chép nguyên văn đoạn code; phần mềm tự tìm vị trí; thất bại thì thử cách khác	AI tự ghi số dòng; phần mềm chỉ "nấn" lệch tối đa 3 dòng	AI hay đếm sai dòng. Cách của OCR bền hơn rõ rệt
Kiểm chứng lại nhận xét	Người kiểm tra chéo mặc định giữ , chỉ bỏ khi có bằng chứng sai	AI được hỏi "chỉ giữ cái nào thật sự là vấn đề"	Cách của HermesQA có thể xóa nhầm nhận xét đúng. Thực nghiệm cho thấy bước này không mang lại lợi ích đo được
Danh sách kiểm tra	Riêng cho 57 loại file, kèm mục "không báo cái gì"	3 bản theo vai, không có mục "không báo"	Thiếu mục "không báo" là nguồn của phần lớn báo động giả
Chọn file cần xem	6 bước lọc cứng: file nhị phân, file chứa mật khẩu, file test, file sinh tự động, file quá lớn...	Chỉ lọc theo danh sách bỏ qua do người dùng khai	HermesQA có thể gửi cả file mật khẩu lên AI
Thay đổi lớn	Chia thành cụm ≤ 10 file, mỗi cụm một phiên AI, có ngân sách chi phí	Cắt bớt âm thầm sau 25 file hoặc 8.000 ký tự mỗi file	Người dùng không biết phần nào đã bị bỏ qua
Đăng kết quả lên GitHub	Nhớ nhận xét cũ, lần sau không đăng lại	Mỗi lần chạy đăng mới	Lập trình viên sửa rồi đẩy tiếp sẽ thấy nhận xét trùng
Lưu vết để điều tra	Ghi mọi câu hỏi và trả lời của AI, chữ ký cấu hình, tiếp tục từ chỗ dở	Báo cáo tổng hợp + cơ sở dữ liệu	Khó truy ngược "vì sao AI nói thế"
Bộ đánh giá	200 PR thật, 1.505 lỗi do 80 kỹ sư xác nhận, chấm bằng AI bỏ phiếu 5 lần	53 PR tiềm lỗi nhân tạo, chấm theo vị trí và loại lỗi	Kết quả của đồ án chưa thể so sánh với số của OCR

Một con số cần nhớ. Trên bộ PR thật của họ, mô hình Claude-4.5-Sonnet dùng theo kiểu agent chỉ đạt F1 khoảng 16% (chính xác 40%, bắt được 10% lỗi). HermesQA đạt F1 67–70% trên lỗi tiềm nhân tạo. Hai số này không đo cùng một thứ: lỗi tiềm rõ ràng và nằm trong thay đổi nhỏ, PR thật thì rối và lỗi ẩn. Khi bảo vệ, nên chủ động nêu con số 16% để hội đồng thấy đồ án hiểu giới hạn của bộ dữ liệu mình tự sinh.

3. Bài học lớn nhất: "lai" không chỉ là "công cụ tĩnh + AI"

Đồ án định nghĩa hệ thống lai là *công cụ tĩnh cộng với AI*. OCR cho thấy một nghĩa thứ hai của "lai": **mọi bước không được phép sai thì giao cho quy tắc cứng, chỉ giao cho AI phần cần phán đoán**. Theo nghĩa đó, HermesQA hiện giao cho AI ba việc mà lẽ ra quy tắc cứng làm tốt hơn: đếm số dòng, quyết định bỏ nhận xét nào, và (gián tiếp) chọn phần nào của thay đổi được đọc.

Các đề xuất dưới đây chuyển ba việc đó về phía quy tắc cứng, giữ nguyên điểm mạnh là sandbox kiểm tra tĩnh.

4. Bảy đề xuất

Mỗi đề xuất có cùng khung: vấn đề hiện tại, OCR làm thế nào, đề xuất cụ thể, cách đo kết quả, công sức ước lượng.

Đề xuất 1 — Ghim nhận xét bằng đoạn code nguyên văn thay vì số dòng

Vấn đề hiện tại. HermesQA yêu cầu AI ghi "lỗi ở dòng 42". AI đếm dòng kém, giống người đọc một trang dài rồi đoán số dòng. Hệ thống chữa cháy bằng cách chấp nhận lệch tối đa 3 dòng; lệch hơn thì nhận xét bị đẩy xuống phần phụ lục, không hiện đúng chỗ.

OCR làm thế nào. AI phải **chép lại nguyên văn** một đến ba dòng code có vấn đề. Phần mềm tìm đoạn đó trong thay đổi bằng so khớp chuỗi (như Ctrl+F). Không thấy thì tìm trong cả file; vẫn không thấy thì thử các file khác; cuối cùng mới hỏi AI lần nữa với yêu cầu "chỉ chép lại, không giải thích".

Đề xuất. Thêm trường "đoạn code" vào định dạng trả lời của AI. Phần mềm so khớp trước, chỉ dùng số dòng AI đưa làm phương án dự phòng.

Cách đo. Bộ thực nghiệm hiện chấm "đúng" khi lệch tối đa 3 dòng. Siết xuống lệch 0 dòng và so sánh tỷ lệ bắt được trước và sau.

Công sức. Khoảng 100 dòng mã, 1–2 ngày kể cả đo.

Đề xuất 2 — Biến bước "tự kiểm chứng" thành "kiểm tra sự thật"

Vấn đề hiện tại. Bước tự kiểm chứng hiện hỏi AI: "chỉ giữ những nhận xét thật sự là vấn đề". Câu hỏi này mời AI xóa bất cứ thứ gì nó cảm thấy không chắc. Thực nghiệm của đồ án đo được bước này tốn gấp đôi chi phí mà không cải thiện kết quả.

OCR làm thế nào. Người kiểm tra chéo được dặn ngược lại: **mặc định giữ**. Chỉ được bỏ nhận xét khi có một trong hai bằng chứng: (A) đoạn code mà nhận xét nói tới không có trong thay đổi, hoặc (B) một dòng trong thay đổi phủ định trực tiếp điều nhận xét khẳng định. Có nhóm nhận xét "được bảo vệ" (an toàn bộ nhớ, đồng thời, thay đổi hành vi) không bao giờ được bỏ dù có nghi ngờ. Lý do họ nêu: giữ một nhận xét sai tốn vài giây của người đọc; bỏ một nhận xét đúng thì lỗi lọt ra ngoài và không ai biết.

Đề xuất. Viết lại lời nhắc kiểm chứng theo đúng triết lý "mặc định giữ, chỉ bỏ khi chứng minh được". Phần "dòng có nằm trong thay đổi không" làm bằng phần mềm, không tốn AI. Giữ danh sách nhóm được bảo vệ.

Cách đo. Chạy lại thí nghiệm bật/tắt kiểm chứng (đã có sẵn cấu hình `11m_noverify`). Nếu bước mới giảm báo động giả mà không làm giảm số lỗi bắt được, đề án có một đóng góp dương thay cho kết quả âm hiện nay.

Công sức. Chủ yếu là viết lại lời nhắc và thêm kiểm tra bằng mã. 1–2 ngày.

Đề xuất 3 — Danh sách kiểm tra theo loại file, kèm mục "không báo"

Vấn đề hiện tại. Ba vai của HermesQA dùng ba bản hướng dẫn chung cho mọi ngôn ngữ. Vai kiểm thử (QA) báo "thiếu unit test cho hàm X" ở 10 trên 35 trường hợp thử nghiệm, dù đó không phải lỗi được tìm. Đây là nhận xét đúng về nguyên tắc nhưng gây nhiễu.

OCR làm thế nào. Có 57 bản danh sách kiểm tra, mỗi loại file một bản (Python, Go, Dockerfile, file cấu hình...). Mỗi bản đều có các mục "**Không báo** khi...", ví dụ: không báo biến không dùng trong file khai báo kiểu, không báo vấn đề đồng thời nếu không có bằng chứng chạy song song. Dòng đầu tiên của mọi bản: "Ưu tiên chính xác hơn bao phủ: chỉ nêu lỗi khi chắc chắn".

Đề xuất. Thêm cơ chế: dựa vào phần mở rộng của file, chọn danh sách kiểm tra phù hợp và đưa vào lời nhắc cùng với hướng dẫn vai. Mỗi danh sách có mục "không báo". Riêng vai QA: không báo "thiếu test" nếu thay đổi không thêm hàm công khai mới. Mặc định bỏ file test ra khỏi vai logic và hạ tầng. OCR dùng giấy phép Apache-2.0 nên có thể tham khảo và trích dẫn các bản này.

Cách đo. Số báo động giả của cấu hình chỉ AI và cấu hình kết hợp trên bộ thực nghiệm; mục tiêu là giảm nhóm "thiếu unit test" và "thiếu đóng kết nối" mà không mất lỗi thật.

Công sức. 2–3 ngày, phần lớn là soạn nội dung danh sách.

Đề xuất 4 — Không cắt bớt âm thầm; có cổng kích thước

Vấn đề hiện tại. Nếu một thay đổi có hơn 25 file, hoặc một file đổi hơn 8.000 ký tự, HermesQA **lặng lẽ cắt bỏ phần dư** và vẫn trả lời như đã xem hết. Đây đúng là lớp lỗi "công cụ im lặng" mà đề án đã tìm ra bốn lần ở chỗ khác.

OCR làm thế nào. Trước khi gọi AI, phần mềm đếm kích thước. File quá lớn bị loại và **ghi rõ lý do** trong kết quả. Thay đổi lớn được chia thành cụm tối đa 10 file, mỗi cụm là một phiên AI riêng, chạy song song. Có lệnh "xem trước" để biết file nào sẽ được xem mà không tốn tiền.

Đề xuất. Bước tối thiểu: khi phải cắt, ghi vào báo cáo "đã bỏ qua N file / M dòng vì vượt ngưỡng". Bước đầy đủ: chia theo file hoặc nhóm file và gọi AI riêng từng nhóm.

Cách đo. Tạo thêm vài trường hợp thử nghiệm có thay đổi lớn (ví dụ 40 file), kiểm tra báo cáo nêu đúng phần bị bỏ, và lỗi nằm ở file thứ 30 vẫn được bắt.

Công sức. Bước tối thiểu nửa ngày; bước đầy đủ 2–3 ngày.

Đề xuất 5 — Danh sách loại trừ mặc định và chặn file chứa bí mật

Vấn đề hiện tại. HermesQA chỉ bỏ qua file do người dùng khai. Nếu thay đổi có file `.env` (thường chứa mật khẩu, khóa API), file khóa phiên bản thư viện, hay file sinh tự động, tất cả được gửi lên AI. Công cụ gitleaks có bắt được bí mật, nhưng bắt xong thì bản diff chứa bí mật đó vẫn đã đi lên dịch vụ AI bên ngoài.

OCR làm thế nào. Có danh sách cứng: không bao giờ gửi `.env`, khóa SSH, `.npmrc` ...; mặc định bỏ `node_modules`, `vendor`, file `.lock`, file `.min.js`, file sinh tự động, ảnh chụp kiểm thử. Bước này chạy trước mọi quy tắc của người dùng và không thể bị ghi đè.

Đề xuất. Thêm hai danh sách mặc định tương tự. File bí mật: loại trước khi gửi AI, và ghi chú trong báo cáo "file X không được xem vì có thể chứa bí mật".

Cách đo. Kiểm thử đơn vị với một thay đổi chứa `.env` và `package-lock.json`: xác nhận chúng không xuất hiện trong lời nhắc gửi AI.

Công sức. Nửa ngày.

Đề xuất 6 — Nhớ nhận xét giữa các lần chạy

Vấn đề hiện tại. Lập trình viên nhận nhận xét, sửa một phần, đẩy tiếp; HermesQA chạy lại và đăng toàn bộ nhận xét lần nữa, kể cả những cái chưa đổi. Người đọc thấy trùng lặp và dần bỏ qua bot.

OCR làm thế nào. Mỗi nhận xét có "dấu vân tay" (file + đoạn code + loại lỗi). Lần chạy sau chỉ đăng nhận xét mới, giữ một bản tóm tắt cố định được cập nhật tại chỗ.

Đề xuất. Lưu dấu vân tay vào cơ sở dữ liệu PostgreSQL đã có; khi đăng, bỏ qua nhận xét đã tồn tại ở lần trước và còn nguyên vị trí.

Cách đo. Mở một PR thử, chạy hai lần liên tiếp, đếm số nhận xét trùng.

Công sức. 1 ngày; cần cấu hình GitHub App thật để thử.

Đề xuất 7 — Nâng cấp cách đánh giá

Vấn đề hiện tại. Bộ chấm điểm hiện tính mọi nhận xét ngoài danh sách lỗi tiềm là "báo động giả", kể cả nhận xét đúng (ví dụ "ảnh cơ sở Docker chưa ghim phiên bản"). Báo cáo đã nêu precision là "cận dưới", nhưng chưa có con số tách bạch giữa "đúng nhưng ngoài danh sách" và "bịa".

OCR làm thế nào. Bài báo AACR-Bench dùng một mô hình AI độc lập làm giám khảo, hỏi "hai nhận xét này có cùng bản chất kỹ thuật không", hỏi 5 lần và lấy đa số. Dữ liệu gốc là PR thật, lỗi do 80 kỹ sư xác nhận chéo.

Đề xuất. (a) Thêm bước giám khảo AI bỏ phiếu để phân loại từng báo động giả thành "đúng nhưng ngoài danh sách" hoặc "bịa thật"; từ đó báo cáo được precision thực. (b) Lấy một phần PR Python từ bộ AACR-Bench (công khai trên Hugging Face) làm bộ dữ liệu thứ ba, có PR thật.

Cách đo. Chính kết quả của bước này là số liệu mới cho mục 7 của báo cáo.

Công sức. (a) 1–2 ngày; (b) 2–3 ngày tùy định dạng dữ liệu.

5. Những gì KHÔNG nên làm theo

Ý tưởng của OCR	Lý do không làm trong khuôn khổ đề án
AI có quyền tự mở file, tìm kiếm, lật toàn bộ mã nguồn (agent có công cụ)	Nhiều tháng công sức, cần cơ chế nén bộ nhớ, giới hạn vòng lặp, xử lý lỗi công cụ. Chính bài báo của họ cho thấy kiểu agent bắt ít lỗi cục bộ hơn cách đọc thẳng thay đổi. Ghi vào "hướng phát triển"

Ý tưởng của OCR	Lý do không làm trong khuôn khổ đề án
Tiện ích cho VS Code, IntelliJ; trình xem phiên làm việc; đo đạc OpenTelemetry	Giá trị sản phẩm, không phải giá trị học thuật
Chế độ "ủy quyền" cho trợ lý lập trình khác chạy review	Phụ thuộc hệ sinh thái bên ngoài, không đo được

6. Thứ tự ưu tiên

Ưu tiên	Đề xuất	Giải quyết điểm yếu nào đã đo được	Công sức	Có đo được bằng bộ thực nghiệm hiện tại không
1	Ghim bằng đoạn code (ĐX1)	Nhận xét lệch dòng, phải nói chấm điểm ± 3 dòng	1–2 ngày	Có
2	Kiểm tra sự thật thay cho tự kiểm chứng (ĐX2)	Bước tự kiểm chứng không có lợi ích đo được	1–2 ngày	Có
3	Danh sách kiểm tra theo loại file + "không báo" (ĐX3)	10/35 báo động giả "thiếu unit test"	2–3 ngày	Có
4	Không cắt âm thầm (ĐX4)	Lớp lỗi "công cụ im lặng"	0,5–3 ngày	Cần thêm trường hợp thử
5	Loại trừ mặc định, chặn bí mật (ĐX5)	Rủi ro gửi bí mật lên AI	0,5 ngày	Kiểm thử đơn vị
6	Nhớ nhận xét giữa các lần (ĐX6)	Trùng lặp khi đẩy tiếp	1 ngày	Cần GitHub App thật
7	Nâng cấp đánh giá (ĐX7)	Precision chỉ là cận dưới; chưa có PR thật	3–5 ngày	Tạo ra số liệu mới

Nếu chỉ còn một tuần: làm 1, 2, 3 và phần tối thiểu của 4, 5. Ba đề xuất đầu biến hai kết quả âm của thực nghiệm (tự kiểm chứng vô ích, báo động giả cao) thành hai thí nghiệm có đối chứng trước/sau, là dạng đóng góp hội đồng đánh giá cao nhất.

7. Thuật ngữ dùng trong tài liệu

Thuật ngữ	Nghĩa đơn giản
Pull request (PR)	Một gói thay đổi mã nguồn mà lập trình viên đề nghị đưa vào sản phẩm, cần người khác duyệt
Diff / thay đổi	Phần khác biệt giữa bản cũ và bản mới của file, chỉ gồm các dòng thêm, xóa, sửa
Công cụ kiểm tra tĩnh	Phần mềm đọc mã nguồn theo quy tắc cố định mà không chạy chương trình, như kiểm tra chính tả
Hộp cát (sandbox)	Môi trường cách ly, không có mạng, không có quyền, để chạy thứ chưa đáng tin

Thuật ngữ	Nghĩa đơn giản
Mô hình ngôn ngữ lớn (LLM, "AI")	Hệ thống như ChatGPT, Claude, Gemini; đọc văn bản và trả lời bằng văn bản
Lời nhắc (prompt)	Văn bản hướng dẫn gửi cho AI trước mỗi câu hỏi
Agent có công cụ	AI được phép tự gọi các thao tác (mở file, tìm kiếm) nhiều vòng trước khi trả lời
Precision (độ chính xác)	Trong những gì hệ thống báo, bao nhiêu phần là lỗi thật
Recall (độ bao phủ)	Trong những lỗi thật có sẵn, hệ thống bắt được bao nhiêu phần
F1	Một con số gộp precision và recall; chỉ cao khi cả hai cùng cao
Báo động giả (false positive)	Hệ thống báo lỗi nhưng không phải lỗi, hoặc không nằm trong danh sách lỗi đã biết
Ground truth	Danh sách lỗi "đáp án" dùng để chấm điểm
Tất định (deterministic)	Cùng đầu vào luôn cho cùng đầu ra; trái với AI có thể trả lời khác nhau mỗi lần
Dấu vân tay nhận xét	Một mã rút gọn đại diện cho nhận xét, để nhận ra "đã thấy cái này rồi"

8. Nguồn đã khảo sát

- Mã nguồn Open Code Review: <https://github.com/alibaba/open-code-review> (giấy phép Apache-2.0). Các phần đã đọc: `README.md`, `ASSURANCE_CASE.md`, `ROADMAP.md`, `internal/agent/` (chọn file, gom nhóm, điều phối), `internal/diff/resolver.go` và `relocation.go` (định vị nhận xét), `internal/config/template/prompts/` (toàn bộ lời nhắc), `internal/config/rules/` (danh sách kiểm tra theo ngôn ngữ), `internal/config/allowlist/` (loại trừ mặc định), `internal/llmloop/` (vòng lặp AI, nén bộ nhớ), `action.yml` và ví dụ GitHub Actions, tài liệu `review-rules.md` và `faq.md`.
- Bài báo AACR-Bench: <https://arxiv.org/abs/2601.19494> — bộ dữ liệu 200 PR thật, 10 ngôn ngữ, 1.505 lỗi do 80 kỹ sư xác nhận; cách chấm bằng AI giám khảo bỏ phiếu 5 lần; kết quả các mô hình (Claude-4.5-Sonnet agent: precision 39,9%, recall 10,1%, F1 16,1%).
- Bộ dữ liệu: <https://huggingface.co/datasets/Alibaba-Aone/aacr-bench>
- Số liệu HermesQA trích từ `BAO_CAO_TIEN_DO.md` mục 7 (cập nhật 05/10/2026).

9. Kết quả sau khi triển khai (cập nhật 06/10/2026)

Cả bảy đề xuất đã được viết thành mã và kiểm thử (commit `d2c5a0b`, chỉnh sửa thêm ở `2133adf`; 34 kiểm thử tự động đều đạt). Phần này ghi lại điều đo được, kể cả điều không như mong đợi.

9.1. Trên bộ lỗi tiềm (35 PR Python) — số sơ bộ

Chạy với cùng mô hình như trước (`gemini-3.1-flash-lite`) để so sánh công bằng. **Lưu ý:** đây là lần chạy đầu của pipeline mới; sau đó chúng tôi phát hiện lỗi ở bước kiểm tra sự thật (mục 9.3) nên cấu hình `llm` sẽ được chạy lại; số dưới đây có thể thay đổi vài điểm.

Cấu hình	Trước (F1)	Sau (F1)	Báo động giả (FP)
Chỉ AI, có kiểm chứng	70,1%	86,5%	27 → 6
Chỉ AI, không kiểm chứng	69,4%	90,7%	28 → 5
Bộ đa ngôn ngữ (18 PR), chỉ AI	73,5%	87,8%	13 → 5

Giải thích cho người không chuyên: điểm tăng chủ yếu vì **danh sách "Không báo"** (đề xuất 3) — AI thôi nhận xét những thứ đúng-nhưng-vô-ích ("thiếu docstring", "nên thêm type hint"...). Đổi lại, chúng tôi phải ghi rõ nguy cơ: danh sách này được viết sau khi đã nhìn thấy bộ lỗi tiềm, nên một phần điểm tăng có thể là "học thuộc đề". Đó chính là lý do phải có mục 9.2.

Việc **ghim nhận xét bằng đoạn code** (đề xuất 1) hoạt động đúng kỹ thuật — 65/65 nhận xét ghim được — nhưng không làm tăng điểm "đúng từng dòng" trên bộ này, vì các PR tiềm lỗi vốn nhỏ và ít lệch dòng. Hiệu quả của nó chỉ thấy được trên PR thật.

9.2. Trên PR thật (15 PR mã nguồn mở, 68 vấn đề do kỹ sư xác nhận) — số đã chốt

Dùng tập con Python của AACR-Bench, chấm theo cách của bài báo (AI giám khảo bỏ 3 phiếu, xác nhận "cùng một vấn đề"). Mô hình: `gemini-3.5-flash-lite` (mô hình cũ hết hạn mức trong ngày; thí nghiệm này không cần so với số cũ).

Cấu hình	Nhận xét đưa ra	Trúng	Precision	Recall	F1
Chỉ AI	8	1	12,5%	1,5%	2,6%
AI + công cụ tĩnh (như cũ)	286	2	0,7%	2,9%	1,1%
AI + công cụ tĩnh, chỉ giữ cảnh báo trong phạm vi thay đổi	43	1	2,3%	1,5%	1,8%

Ba điều rút ra:

- Khoảng cách giữa "đề thi" và "đời thật" là rất lớn.** Trên lỗi tiềm, hệ thống bắt được ~90% vấn đề; trên PR thật chỉ 1,5–3%. Để so sánh, hệ thống tốt nhất trong bài báo AACR-Bench (agent dùng Claude 4.5 Sonnet) cũng chỉ đạt recall 10,1%. Con số của chúng tôi thấp hơn vì AI gần như **im lặng** trên code thật: 15 PR chỉ sinh 8 nhận xét, không phải vì bị bộ lọc chặn (bước kiểm tra sự thật không loại nhận xét nào) mà vì mô hình nhỏ cộng với danh sách "Không báo" khiến nó quá dè dặt. Đây là điểm cần nói thẳng trong báo cáo thay vì chỉ trình bày con số 86–90%.
- Công cụ tĩnh trên repo thật chủ yếu báo về code không hề thay đổi.** 240/275 cảnh báo tĩnh nằm ngoài phần sửa của PR. Chúng vừa làm phiền người đọc, vừa không gửi được (GitHub không cho đặt comment review lên dòng ngoài phạm vi thay đổi). Chúng tôi đã thêm bộ lọc "chỉ trong phạm vi thay đổi": số cảnh báo giảm 286 → 43, mất đúng 1 lần trúng (vốn cũng không đăng được).
- Chuẩn bị dữ liệu thật dễ sai hơn tưởng.** Lần chạy đầu, chúng tôi hiểu ngược hai trường commit của AACR-Bench (tên trường `pr_source_commit` thực ra là nhánh đích), nên hệ thống đã review diff *đảo chiều* hoặc kèm hàng nghìn dòng không thuộc PR. Phát hiện nhờ đối chiếu số dòng với dữ liệu gốc và GitHub API; toàn bộ lần chạy đó bị bỏ. Bài học: luôn kiểm tra kích thước diff trước khi tin kết quả.

9.3. Lỗi tìm thấy nhờ đo đạc và đã sửa

- **Kiểm tra sự thật "lý luận vòng".** AI giám khảo từng loại 4 nhận xét *đúng* với lý do "code đã xử lý rồi", nhưng "bằng chứng" nó dẫn chính là dòng đang bị báo lỗi (ví dụ nhận xét "dùng `== None`" bị loại với bằng chứng `if user == None:`). Nay phần mềm bác mọi bằng chứng trùng dòng bị báo — căn cứ B phải là một dòng **khác** đã xử lý vấn đề.
- **Hạn mức theo phút và JSON hỏng.** Trên PR thật, mô hình trả về JSON có xuống dòng thật hoặc thiếu dấu phẩy (~1/15 PR), và dịch vụ giới hạn 15 lượt/phút. Hệ thống nay chấp nhận xuống dòng trong chuỗi, gọi lại một lần khi JSON hỏng, và tự đợi đúng thời gian được báo khi bị giới hạn.

9.4. Việc còn lại

- Chạy lại cấu hình `11m` và `both` trên hai bộ lỗi tiềm với bản sửa 9.3 (chờ hạn mức ngày của mô hình), chấm ở độ lệch 0 dòng, và chạy giám khảo phân loại báo động giả (đề xuất 7) — nên dùng một mô hình khác làm giám khảo để tránh "tự khen".
- Thử mô hình mạnh hơn trên bộ PR thật để biết phần "im lặng" đến từ mô hình hay từ lời nhắc.